Lab 03: Pardon the Interruptions

ECE 484: Embedded Systems Design

Grade	Score	Signature
Attendance: Gabe Litteken		
Attendance: James Nippa		
Attendance:		
Demonstration		

Overview

In this lab, you will design a system that will display the number of button presses on two seven-segment displays. The two seven-segment displays will show the 8-bit hex equivalent of the total number of button presses and a separate push button will be included to clear the count at any time. Oh, and did I mention, **you are not allowed to write any code in the `while(1)` loop**?

Deliverables

1. System Demonstration (10 points)
2. Completed Worksheet submitted to Canvas (5 points)
3. Commented Source Code submitted to Canvas (5 points)

Instructions

Your system will utilize two push buttons and two seven-segment displays. First, you will need to construct a pull-up circuit for each of your push buttons. For the push button that will increment the count, you will connect the middle node of the corresponding pull-up circuit to INT0 (PIND2) on your microcontroller. The push-button that will clear the count will have the middle node of its corresponding pull-up circuit connected to a GPIO of your choice. Next, you will need to include two seven-segment displays in your system. Just like the previous lab, you will need seven current limiting resistors that will connect to the anodes of both seven segment display and the current limiting circuits will be connected to PORTD. However, you will not be able to use PIND2 (INT0) so you will need to connect the seven segments to the remaining pins on PORTD.

In this lab, you will utilize a concept known as time mulitplexing to display both seven-segments "simultaneously". In reality, you will be taking advantage of the flicker fusion threshold (FFT) of the human eye to quickly strobe the two seven-segments independently such that your eye will perceive both seven-segments as being constantly lit. The reason you need to utilize time multiplexing is due to the current sourcing limitations of your microcontroller. Looking at the datasheet for the ATmega328P (Section 29: Typical Characteristics), Figure 29-10 shows that the source voltage drops as an I/O pin sources more current. When sourcing 20mA out of an I/O pin, the source voltage is below $\approx$ 4.6V which will lead to dimmer LEDs and puts extra stress (i.e.

Scanned with CamScanner

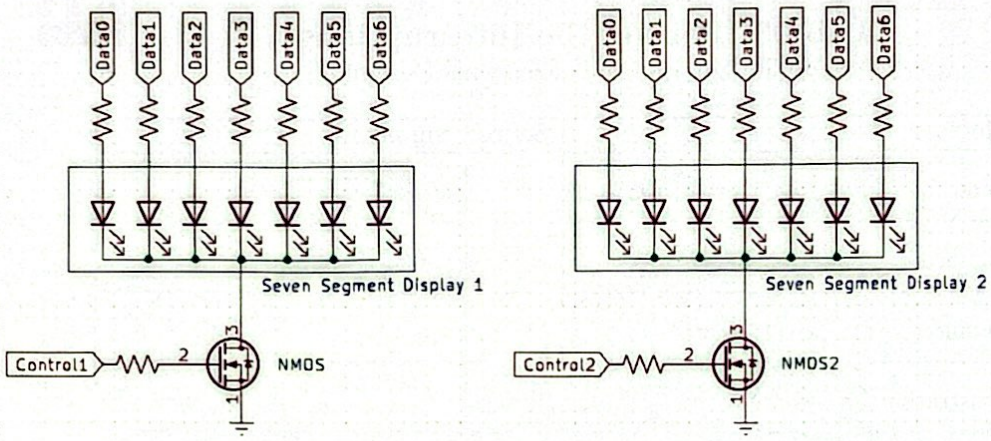


Figure 1: Seven-Segment Displays Schematic

increased heat dissipation) on the internal 5V regulator inside you microcontroller. Any unexpected current spikes could further reduce the source voltage which may cause the 5V regulator to fail. To avoid this issue, we will use MOSFET drivers to quickly flicker the seven-segment displays.

The seven-segment driver circuitry, shown in Figure 1, will utilize NMOS transistors to drive the cathodes of each seven-segment. A pinout for th NMOS transistor is provided in the ZVN3306A datasheet on Canvas. The drain (D) of each transistor will connect to the common cathode pins of each seven-segment while the source (S) will connect to ground. The gate (G) of each transistor will be connected to a current limiting resistor (10kΩ) and a GPIO pin of your choice. When 0V is supplied to the gate, the transistor is completely OFF such that the transistor acts as an open switch which turns off the seven-segment display. Conversely, when +5V is supplied to the gate, the gate is saturated such that the transistor is completely ON and current flows through the transistor illuminating the seven-segment display.

Once you have constructed your circuit, your code will first need to configure the Timer 1 module in your microcontroller to trigger an output compare interrupt. You will calculate OCR1A values that correspond to interrupts that trigger every 500ms, 50ms, and 10ms. Make sure to record these calculations in your Question 1 at the end of this lab manual. The Timer 1 ISR will choose which seven-segment display to illuminate and then will pass the corresponding upper or lower 4 bits of the current button count to the `setSevenSegments` lab you used in the previous lab. You will need to make some modifications to `setSevenSegments` to incorporate the fact that PIND2 is being used by a push button. You will also need to configure the external interrupt (INT0) for the increment count push button and a pin change interrupt for the clear count push button. After each interrupt is configured, you will need to add the proper functionality to the respective ISR. **You are not permitted to write any code inside the `while(1)` loop. Doing so will result in a 0 for the demonstration.**

Scanned with CamScanner

Demonstration Rubric

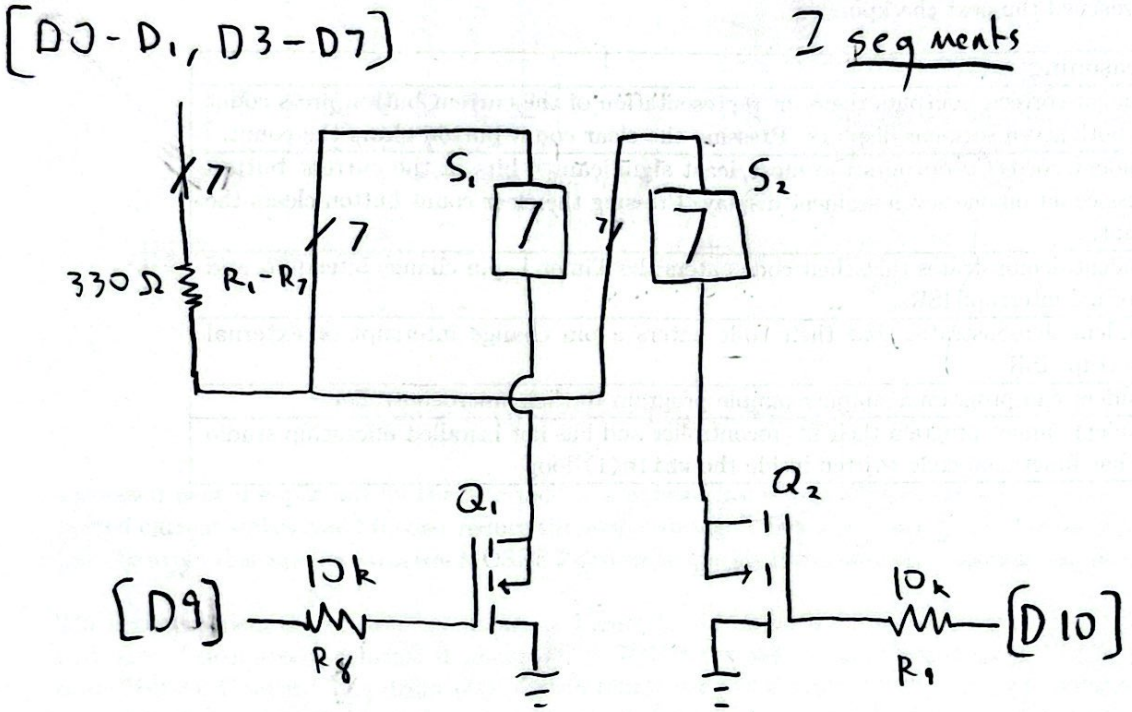
Please demonstrate your system to Dr. Swanson or the TA and they will record your demonstration score at the top of your lab manual. Checkpoint scores are provided in the following rubric. Scores between checkpoints may be awarded if a student has exceeded the previous checkpoint but has not reached the next checkpoint(s).

Score	Reasoning
10.0	Student correctly outputs the 8-bit representation of the current button press count on both seven segment displays. Pressing the clear count button clears the count.
8.0	Student correctly outputs the most/least significant 4-bits of the current button press count on one seven segment display. Pressing the clear count button clears the count.
6.0	Student demonstrates that their code enters the Timer 1, pin change interrupt, and external interrupt ISRs.
4.0	Student demonstrates that their code enters a pin change interrupt or external interrupt ISR.
2.0	Student can program a simple example program to their microcontroller.
0.0	Student cannot program their microcontroller and has not installed microchip studio or has functional code written inside the `while(1)` loop

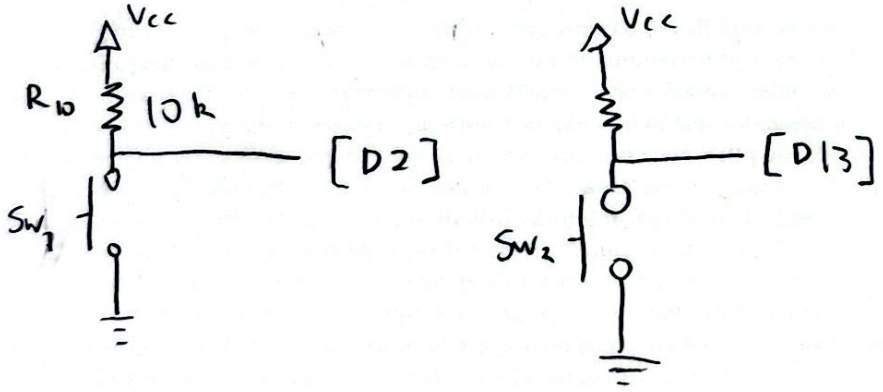
Scanned with CamScanner

Circuit Diagram

Draw a circuit diagram for your system. Make sure to label all major modules, resistance values, and signal lines.



buttons



Scanned with CamScanner

Design Questions

1. Include your calculations for the OCR1A values that correspond to the following sampling intervals. Make sure to include which clock prescaler you used.

1. 500 ms

Prescaler : 256

$(0.5 \cdot \frac{16e6}{256}) - 1 = \boxed{31250}$

2. 50 ms

prescaler : 64

$(50e-3 \cdot \frac{16e6}{64}) - 1 = \boxed{12500}$

3. 10 ms

prescaler : 8

$(10e-3 \cdot \frac{16e6}{8}) - 1 = \boxed{19999}$

2. Using the ATmega328P datasheet, write the configuration code for Timer 2 such that it operates in CTC mode and triggers an interrupt every 30ms. Make sure to include your OCR2A calculations and explain your choice for a clock prescaler.

$(\Delta T \cdot \frac{Clk}{N}) - 1 = OCR2A$

$(30e-3 \cdot \frac{16e6}{1024}) - 1 = 467.75 \rightarrow$ not possible with Timer 2

TCCR2A = 0b00000001;

TCCR2B = 0b00000111;

TIMSK2 = 0b00000010;

* cant set OCR2A because 467.75 > 255

Scanned with CamScanner

3. Using the ATmega328P datasheet, write the configuration code for Timer 0 such that it operates in normal mode, with a clock prescaler of 64, and triggers an interrupt when the timer overflows.

TCCR0A = 0b00000000;
COM0A COM0B WGM0(1-0)

TCCR0B = 0b00000011;
FOC0 WGM02 CS0(0-2) = 64

TIMSK0 = 0b00000001;
OCIE0B OCIE0A TOIE0

4. The flicker fusion threshold of the human eye is ≈ 60Hz. Lights that flicker at this rate appear steady to the human eye. Calculate the OCR1A value that corresponds to the flicker fusion threshold. Since OCR1A values need to be whole numbers, should you round your answer up or down? Why?

60 Hz = 16.66 ms

$$\left(16.66e-3 \cdot \frac{16e6}{8}\right) - 1 = 33332.33$$

$$\boxed{33332}$$

We round down to increase the frequency to ~ 60.001 Hz

Scanned with CamScanner